

Jana Hazem Mohamed Taha

Systems and Biomedical Engineering Department

Faculty of Engineering, Cairo University

Third Year

Technical Questions

1.

Generative models learn how to create new data, while discriminative models learn how to tell classes or categories apart. For example, GPT is a generative model because it can generate text. A spam classifier is a discriminative model because it decides whether an email is spam or not.

2.

Transformers are used in modern AI because they can handle long sequences and understand the relationship between different parts of the input. They use attention to decide which words or tokens are important to each other, and this lets the model process many parts of the input efficiently.

3.

Supervised learning uses labeled data, so the model learns from input-output examples.

Unsupervised learning uses data without labels and tries to find patterns or groups in it.

Reinforcement learning works through actions and rewards, where the model learns which actions give better results.

4.

Tokens are the small pieces that text is split into before it is given to an LLM. A token can be a word, part of a word, or sometimes a symbol. They are important because the model reads and generates text in tokens, and the number of tokens also affects the context limit and cost.

5.

Attention helps a neural network focus on the parts of the input that matter most for the current task. For example, while reading a sentence, the model can give more attention to words that are related to the word it is trying to understand. This helps it keep better context and improve the final output.

Technical Questions

6.

Prompt engineering means writing the prompt in a clear way so the model understands the task, the context, and the expected output. It is important because a better prompt usually gives more useful and consistent results. Common types are zero-shot prompts, few-shot prompts, role-based prompts, instruction prompts, and structured prompts.

7.

RAG combines a search step with the language model. First, the system receives the question and searches a knowledge base for relevant information. The useful parts are added to the prompt, then the LLM generates the answer using that retrieved information. This helps the model answer from specific documents instead of depending only on what it learned during training.

8.

An embedding is a numerical representation of text that captures its meaning. Similar pieces of text have embeddings that are close to each other. In retrieval, the system can compare the embedding of the user question with document embeddings and find the most relevant information.

9.

Temperature controls how random the model output is. A low temperature gives more predictable and focused answers. A higher temperature gives more variety and can be useful for creative tasks, but it can also make the answer less consistent.

10.

Chain-of-thought prompting asks the model to work through a problem step by step instead of jumping directly to the final answer. This can help with tasks that need several reasoning steps because it gives the model a clearer structure for solving the problem.

Technical Questions

11.

RAG is mainly used when the information can change or is stored in external documents. Fine-tuning changes the model itself by training it on a specific dataset. For example, I would use RAG for an assistant that answers from CURT documents. I would use fine-tuning when I want the model to learn a specific style, format, or task from many examples.

12.

Before retrieval, I would improve the documents first by cleaning the data, removing repeated content, choosing a good chunk size, adding useful metadata, and using a good embedding model. I can also improve the search with hybrid retrieval, where keyword search and vector search are combined. After retrieval, I can rerank the results, remove weak or repeated chunks, and pass only the most relevant information to the LLM.

13.

I would look at the type of task first, then compare accuracy, latency, cost, context window, privacy, and reasoning ability. I would also consider whether the model can run with the available hardware or through an API. For a system like CURT, privacy and access control are also important because the documents may be internal.

14.

I would check whether the important claims in the answer are supported by the retrieved documents. I can compare the answer with the source text and measure things like citation correctness, factual accuracy, and how much of the answer is actually supported. I would also test the system on a set of questions with known answers.

15.

Computer vision can give Generative AI visual information from images or video. For example, a vision model can detect a part, read a dashboard, or identify an object, and then a language model can explain the result or answer questions about it. This can be useful for inspection, documentation, and helping teams understand visual data.

16.

I would build the assistant as a pipeline so each step has a clear job:

User question -> Login and permissions check -> Question routing -> Document search / Structured data search -> Retrieve and rank useful results -> LLM generates answer -> Source check and citations -> Final answer

1. Login and permissions check

The system first checks who the user is and which team documents they are allowed to access. This prevents one team from seeing another team's private files.

2. Question routing

The system decides whether the question needs document search, structured data, or both. For example, a question about a design report needs document retrieval, while a question about a stored measurement may need structured data.

3. File and data processing

Documents such as PDF, Word, Excel, and text files are cleaned and converted into a common format. Tables and useful metadata should also be kept because they can be important for retrieval.

4. Search and retrieval

The question is converted into an embedding and used to search the knowledge base. I would also use keyword search when exact terms, part numbers, or names are important.

5. Ranking the results

The retrieved chunks are ranked so the most useful information is sent to the LLM. This reduces unrelated context and helps the model focus on the right documents.

6. LLM generation

The LLM gets the user question together with the selected information and generates a direct answer. The prompt should tell it to stay within the provided information and say when the answer is not available.

7. Source check and citations

The system checks that the important claims are supported by the retrieved content and then shows the document names or other sources used for the answer.

8. Final answer

The user receives the answer together with its sources. The system can also log the question, retrieval results, and feedback for later evaluation and improvement.

Main challenges and solutions

Different file formats: Use separate parsers for PDF, Word, Excel, and text, then convert the content into one common structure before indexing it.

Inconsistent or noisy data: Clean duplicate, incomplete, and badly formatted data before retrieval. Keep metadata such as team, file name, date, and document type.

Large documents and context limits: Split documents into meaningful chunks, keep overlap when needed, and retrieve only the most relevant chunks instead of the whole file.

Poor retrieval: Use better embeddings, hybrid search, metadata filters, and reranking to improve the results.

Wrong generated answers: Tell the LLM to use only the retrieved information, require citations, and evaluate answers against a prepared test set.

Unauthorized access: Apply permissions before retrieval, not only after generation, so restricted documents are never included in the model context.

Evaluation

I would create a test set of real CURT-style questions with expected answers and correct source documents. I would measure retrieval quality, answer accuracy, citation correctness, response time, and how often the system refuses to answer when the documents do not contain enough information. I would also test access control with users from different teams to make sure restricted data never appears in their results.